

Bases tecnológicas para el servicio

MATERIA: PROGRAMACIÓN EN JAVA

ACTIVIDAD: – Estudio de Caso: Tienda Departamental “Dreams”

ALUMNO: CESAR CEDILLO
RODRÍGUEZ

GRUPO: 8

DOCENTE: JUAN MANUEL
TOLENTINO MORALES

1. Identificación de Clases y Objetos

De acuerdo con las necesidades descritas para la tienda departamental “Dreams”, se identificaron las clases Producto, Stock, Cliente, Venta, Entidad (clase abstracta) y Calculable (interfaz). Estas clases representan los elementos fundamentales para gestionar productos, clientes y ventas dentro del sistema.

2. Definición de atributos y métodos

A continuación, se describen los atributos y métodos principales de cada clase.

3. Relaciones entre clases

Las clases mantienen relaciones adecuadas con el problema del sistema de tienda. Producto contiene un Stock mediante composición, estableciendo una relación de dependencia total donde el stock no puede existir sin el producto. Venta se asocia tanto a Cliente como a ItemVenta, representando una transacción completa del sistema. Producto y Cliente heredan de Entidad, compartiendo el atributo nombre y estableciendo una jerarquía común para las entidades principales del sistema.

ItemVenta actúa como intermediario entre Producto y Venta, encapsulando la información de cantidad y producto específico dentro de una venta. Tanto Venta como ItemVenta implementan la interfaz Calculable, asegurando que puedan calcular el monto total de sus respectivas operaciones de forma polimórfica.

MenuPrincipal orquesta todas las relaciones del sistema, gestionando colecciones de Producto, Cliente y Venta, mientras utiliza ValidarEntrada para asegurar la integridad de los datos de entrada. Main establece el punto de entrada del sistema creando una instancia de MenuPrincipal e iniciando la aplicación.

La clase ValidarEntrada proporciona servicios utilitarios que son utilizados por MenuPrincipal mediante dependencia, centralizando la lógica de validación y promoviendo la reutilización de código en todo el sistema.

CLASE: Entidad
(Abstracta)

Elemento	Descripción
Tipo	Clase Abstracta
Atributos	String nombre
Constructor	Entidad(String nombre)
Métodos	getNombre() : String
Propósito	Clase base para todas las entidades del sistema

CLASE:
Calculable
(Interfaz)

Elemento	Descripción
Tipo	Interfaz
Atributos	Ninguno
Constructor	No aplica
Métodos	calcularTotal() : double
Propósito	Define contrato para objetos que pueden calcular su total

CLASE: Cliente

Elemento	Descripción
Tipo	Clase Concreta
Atributos	String correoCliente
Constructor	Cliente(String nombreCliente, String correoCliente)
Métodos	getCorreoCliente() : String
Propósito	Representa un cliente del sistema de tienda

CLASE:
Producto

Elemento	Descripción
Tipo	Clase Concreta
Atributos	double precioProd, String categoriaProd, Stock stock
Constructor	Producto(String nombre, double precioProd, String categoriaProd, int cantidadStock)
Métodos	getPrecioProd() : double, getStock() : Stock
Propósito	Representa un producto en el sistema de tienda

CLASE: Stock

Elemento	Descripción
Tipo	Clase Concreta
Atributos	int cantidad
Constructor	Stock(int cantidad)
Métodos	getCantidad() : int, reducirStock(int n) : void, agregarStock(int n) : void, establecerStock(int nuevaCantidad) : void

Propósito	Gestiona la cantidad disponible de un producto
-----------	--

CLASE:
ItemVenta

Elemento	Descripción
Tipo	Clase Concreta
Atributos	Producto producto, int cantidad
Constructor	ItemVenta(Producto producto, int cantidad)
Métodos	getProducto() : Producto, getCantidad() : int, calcularTotal() : double
Propósito	Representa un item dentro de una venta con producto y cantidad

CLASE: Venta

Elemento	Descripción
Tipo	Clase Concreta
Atributos	ArrayList<ItemVenta> carrito, Cliente cliente
Constructor	Venta(Cliente cliente)
Métodos	agregarProducto(Producto producto, int cantidad) : void, calcularTotal() : double, realizarVenta() : void, mostrarCarrito() : void, getCarrito() : ArrayList<ItemVenta>, getCliente() : Cliente
Propósito	Gestiona una venta completa con carrito de compras y cliente

CLASE:
ValidarEntrada
(Utilidad)

Elemento	Descripción
Tipo	Clase Utilitaria
Atributos	Ninguno
Constructor	No aplica (métodos estáticos)
Métodos	leerEntradaConValidacion(Scanner scanner, int min, int max) : int, leerTextoNoVacio(Scanner scanner, String mensaje) : String, leerPrecio(Scanner scanner, String mensaje) : double
Propósito	Centraliza la validación de entradas del usuario

CLASE:
MenuPrincipal

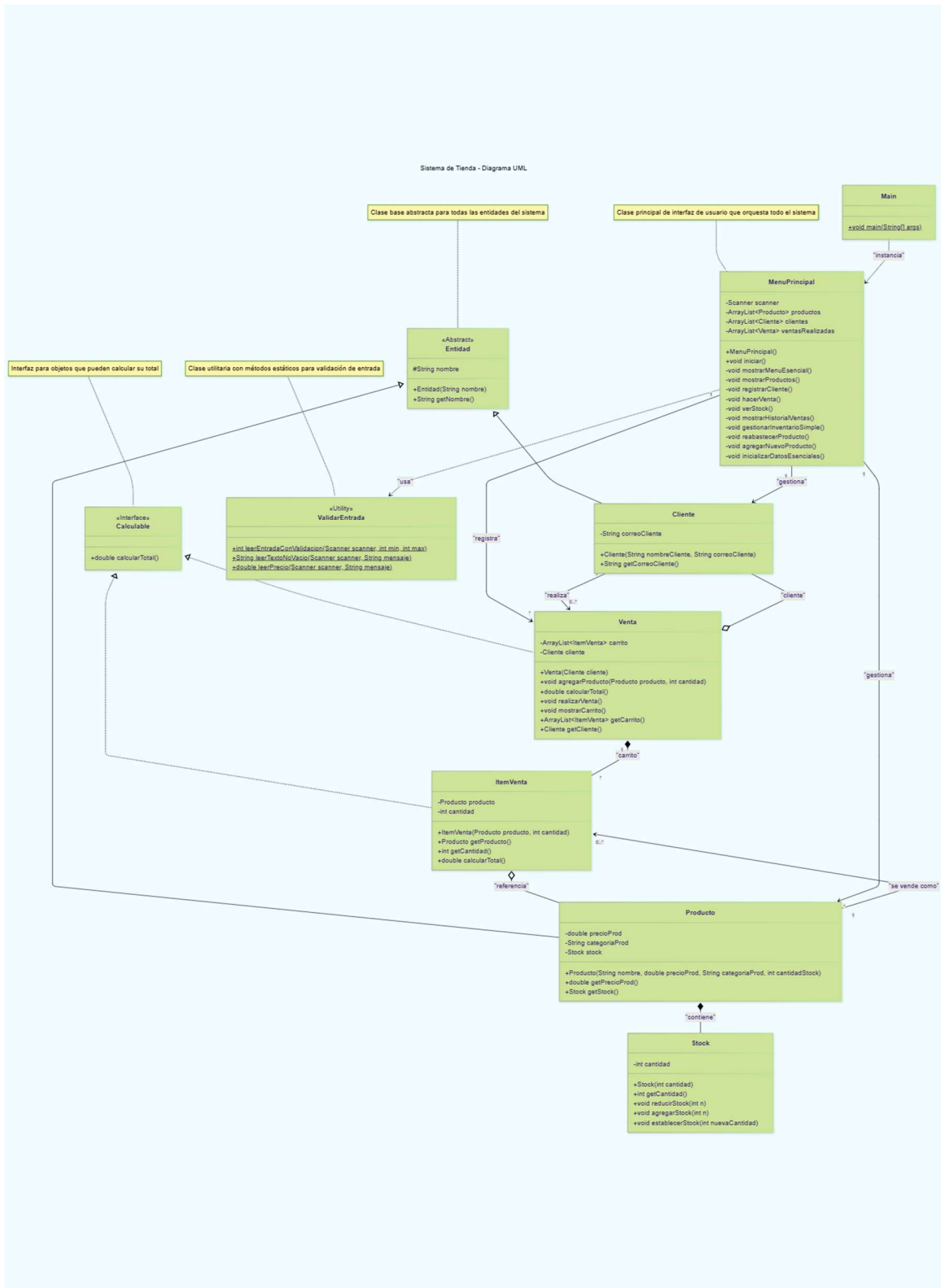
Elemento	Descripción
----------	-------------

Tipo	Clase Concreta
Atributos	Scanner scanner, ArrayList<Producto> productos, ArrayList<Cliente> clientes, ArrayList<Venta> ventasRealizadas
Constructor	MenuPrincipal()
Métodos	iniciar() : void, mostrarMenuEsencial() : void, mostrarProductos() : void, registrarCliente() : void, hacerVenta() : void, verStock() : void, mostrarHistorialVentas() : void, gestionarInventarioSimple() : void, reabastecerProducto() : void, agregarNuevoProducto() : void, inicializarDatosEsenciales() : void
Propósito	Interfaz principal del usuario que orquesta todo el sistema

CLASE: Main

Elemento	Descripción
Tipo	Clase Principal
Atributos	Ninguno
Constructor	No aplica
Métodos	main(String[] args) : void
Propósito	Punto de entrada de la aplicación

Diagrama UML de las Clases de la tienda



Ejemplo de funcionamiento del programa en la terminal

=== SISTEMA DE TIENDA - MENU PRINCIPAL ===

--- MENU PRINCIPAL ---

1. Ver Productos
2. Nuevo Cliente
3. Hacer Venta
4. Ver Stock
5. Historial Ventas
6. Restock/Nuevo Producto
7. Salir

Opcion: 2

=== REGISTRO DE CLIENTE ===

Nombre del cliente: juan in

Email del cliente: juan@in.com

:) Cliente juan in registrado exitosamente!

--- MENU PRINCIPAL ---

1. Ver Productos
2. Nuevo Cliente
3. Hacer Venta
4. Ver Stock
5. Historial Ventas
6. Restock/Nuevo Producto
7. Salir

Opcion: 3

=== REALIZAR VENTA ===

Clientes disponibles:

1. Cliente Ejemplo

2. juan in

Seleccione cliente (número): 2

[CARRITO] PARA: juan in

=== PRODUCTOS DISPONIBLES ===

1. Laptop - \$15000.0 (Disponibles: 3)

2. Mouse - \$500.0 (Disponibles: 10)

3. Teclado - \$2000.0 (Disponibles: 5)

Opciones:

1-3. Agregar producto al carrito

0. Finalizar compra

Seleccione opción: 3

Cantidad (máximo 5): 2

Producto agregado al carrito.

:) Teclado x2 agregado al carrito!

=== PRODUCTOS DISPONIBLES ===

1. Laptop - \$15000.0 (Disponibles: 3)

2. Mouse - \$500.0 (Disponibles: 10)

3. Teclado - \$2000.0 (Disponibles: 3)

--- CARRITO ACTUAL ---

Que hay en el carrito:

- Teclado: 2 unidad(es) x \$2000.0 = \$4000.0

Subtotal: \$4000.0

Opciones:

1-3. Agregar producto al carrito

0. Finalizar compra

Seleccione opción: 0

=== RESUMEN FINAL ===

Cliente: juan in

Que hay en el carrito:

- Teclado: 2 unidad(es) x \$2000.0 = \$4000.0

[\$] TOTAL: \$4000.0

¿Confirmar venta? (s/n): s

Total comprado del cliente juan in: \$4000.0

:) Venta realizada exitosamente!

[EMAIL] Recibo enviado a: juan@in.com

--- MENU PRINCIPAL ---

1. Ver Productos

2. Nuevo Cliente

3. Hacer Venta

4. Ver Stock

5. Historial Ventas

6. Restock/Nuevo Producto

7. Salir

Opcion: 1

=== PRODUCTOS DISPONIBLES ===

1. Laptop - \$15000.0 (Disponibles: 3)

2. Mouse - \$500.0 (Disponibles: 10)

3. Teclado - \$2000.0 (Disponibles: 3)

--- MENU PRINCIPAL ---

1. Ver Productos
2. Nuevo Cliente
3. Hacer Venta
4. Ver Stock
5. Historial Ventas
6. Restock/Nuevo Producto
7. Salir

Opcion: 4

=== ESTADO DEL STOCK ===

Laptop: 3 unidades :) Disponible

Mouse: 10 unidades :) Disponible

Teclado: 3 unidades :) Disponible

--- Ventas del día: 1 ---

--- MENU PRINCIPAL ---

1. Ver Productos
2. Nuevo Cliente
3. Hacer Venta
4. Ver Stock
5. Historial Ventas
6. Restock/Nuevo Producto
7. Salir

Opcion: 5

=== HISTORIAL DE VENTAS ===

1. Cliente: juan in - Total: \$4000.0

:) Total vendido hoy: \$4000.0

--- MENU PRINCIPAL ---

1. Ver Productos
2. Nuevo Cliente
3. Hacer Venta
4. Ver Stock
5. Historial Ventas
6. Restock/Nuevo Producto
7. Salir

Opcion: 6

=== RESTOCK / NUEVO PRODUCTO ===

1. Reabastecer producto existente
2. Agregar nuevo producto
3. Volver al menu principal

Opcion: 1

--- PRODUCTOS ACTUALES ---

=== PRODUCTOS DISPONIBLES ===

1. Laptop - \$15000.0 (Disponibles: 3)
2. Mouse - \$500.0 (Disponibles: 10)
3. Teclado - \$2000.0 (Disponibles: 3)

Seleccione producto a reabastecer (numero): 3

Stock actual de Teclado: 3

Cantidad a agregar: 2

:) Stock actualizado! Teclado ahora tiene 5 unidades.

--- MENU PRINCIPAL ---

1. Ver Productos
2. Nuevo Cliente

3. Hacer Venta

4. Ver Stock

5. Historial Ventas

6. Restock/Nuevo Producto

7. Salir

Opcion: 6

=== RESTOCK / NUEVO PRODUCTO ===

1. Reabastecer producto existente

2. Agregar nuevo producto

3. Volver al menu principal

Opcion: 2

--- NUEVO PRODUCTO ---

Nombre del producto: CD

Precio del producto: \$10

Categoria del producto: audio

Cantidad inicial en stock: 50

:) Producto 'CD' agregado exitosamente!

--- MENU PRINCIPAL ---

1. Ver Productos

2. Nuevo Cliente

3. Hacer Venta

4. Ver Stock

5. Historial Ventas

6. Restock/Nuevo Producto

7. Salir

Opcion: 60

Opción no válida. Intente de nuevo: cinco

Entrada inválida. Por favor ingrese un número: 7

=== Gracias por usar el Sistema de Tienda ===

PS C:\Users\César\Downloads\clases_tienda (1)>

4. Aplicación de los principios de la Programación Orientada a Objetos

En el sistema se aplican los principios básicos de la Programación Orientada a Objetos, lo que ayuda a que el código sea más ordenado y fácil de mantener. El encapsulamiento se usa para proteger la información interna de las clases. Por ejemplo, en la clase Producto el precio y el stock son atributos privados, por lo que no se pueden modificar directamente desde fuera. Algo similar pasa en las clases Cliente y Stock, donde los datos importantes solo se manejan mediante métodos definidos.

La herencia se aplica a través de la clase abstracta Entidad, de la cual heredan Cliente y Producto. Gracias a esto, ambas clases comparten atributos comunes como el nombre, evitando repetir código. El polimorfismo se puede ver en la interfaz Calculable, ya que tanto Venta como ItemVenta implementan el método calcularTotal(), pero cada uno lo hace de forma diferente según su función. Por último, la abstracción se refleja en el uso de clases abstractas e interfaces, que permiten definir comportamientos generales sin entrar en los detalles de su implementación.

5. Correspondencia entre el análisis, el diseño y la implementación

En general, las clases que se implementaron sí corresponden con lo que se había planeado en el análisis y el diseño del sistema. Se desarrollaron las clases principales como Cliente, Producto y Venta, junto con sus relaciones y funciones básicas, como el manejo del inventario y el cálculo del total de una venta. Durante la implementación hubo cambios que no había considerado, como agregar validaciones y crear el menú principal, para que el sistema funcionara mejor, el menú principal me tomó más tiempo del que pensaba, primero contenía validaciones pero sentí que ya era una clase muy saturada y pasé las validaciones a una clase separada .

5. Encapsulamiento en Producto y Venta

En la clase Producto, el encapsulamiento sirve para proteger datos importantes como el precio y el stock, permitiendo que solo se acceda a ellos mediante métodos. De la misma forma, en la clase Venta el carrito de compras y el cliente están encapsulados, lo que ayuda a mantener el control sobre cómo se usa esa información. Esto evita errores y hace que el sistema sea más fácil de mantener.

6. Relación entre Venta y productos

La relación entre Venta y los productos es de agregación, ya que los productos existen por separado y no dependen de una venta para seguir existiendo. Una venta solo guarda referencias a los productos a través de ItemVenta, por lo que si se elimina una venta, los productos continúan en el sistema. Por esta razón, no se trata de una relación de composición.

7 Función de la clase utilitaria ValidarEntrada

La clase ValidarEntrada se encarga de centralizar la validación de los datos que ingresa el usuario al sistema. Al utilizar métodos estáticos, sus funciones pueden reutilizarse fácilmente en distintas partes del programa sin necesidad de crear objetos. Esto ayuda a mantener el código más ordenado y consistente.

Esta clase valida principalmente tres tipos de entradas: números enteros dentro de un rango, texto no vacío y precios. En el caso de los menús, se asegura de que el usuario ingrese valores válidos y evita errores cuando se introducen datos incorrectos. También impide que se registren nombres vacíos o con solo espacios y garantiza que los precios sean valores numéricos positivos. Gracias a esto, se reducen errores y la lógica de validación se mantiene en un solo lugar, facilitando el mantenimiento del sistema.

8 Función de la clase MenuPrincipal

La clase MenuPrincipal funciona como el punto central del sistema y se encarga de coordinar todas las acciones principales. Su responsabilidad es mostrar los menús, recibir las opciones del usuario y dirigir el flujo del programa hacia las funcionalidades correspondientes, como registrar clientes, realizar ventas o gestionar el inventario.

Además, esta clase mantiene el estado del sistema mediante **listas** de productos, clientes y ventas, lo que permite manejar correctamente las relaciones entre los objetos. En términos de diseño, MenuPrincipal sigue el patrón **MVC**, donde actúa como controlador, conectando la vista (la consola) con el modelo (las clases de datos). Este enfoque permite un flujo claro, facilita el mantenimiento y hace más sencillo agregar nuevas funciones en el futuro.

9 Patrón de diseño MVC y su aplicación en MenuPrincipal

El patrón de diseño **MVC (Modelo-Vista-Controlador)** es una forma de organizar un sistema separando responsabilidades. El **Modelo** se encarga de los datos y la lógica del negocio, la **Vista** muestra la información al usuario y el **Controlador** actúa como intermediario entre ambos, recibiendo las acciones del usuario y decidiendo qué hacer con ellas.

En este sistema, la clase **MenuPrincipal** sigue este patrón porque cumple principalmente el rol de **controlador**. Esta clase no almacena la lógica interna de los datos, sino que coordina

las acciones del usuario, muestra los menús en consola y llama a los métodos correspondientes de las clases del modelo, como *Producto*, *Cliente* y *Venta*. Por ejemplo, cuando el usuario selecciona una opción del menú, MenuPrincipal decide si se debe registrar un cliente, realizar una venta o mostrar el inventario.

Las clases **Producto**, **Cliente**, **Venta** y **Stock** funcionan como el **Modelo**, ya que contienen los datos y la lógica principal del sistema, como el manejo de precios, stock y cálculos de totales. La **Vista** está representada por la salida en consola, donde se muestran menús, mensajes y resultados al usuario mediante System.out.println.

Gracias a esta separación, el sistema es más claro y fácil de mantener, ya que la clase MenuPrincipal se enfoca en el flujo del programa y la interacción con el usuario, mientras que las demás clases se encargan de manejar la información y las reglas del negocio.